Activity No. 7	
SORTING ALGORITHMS: BUBBLE, SELECTION, AND INSERTION SORT	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed: 16/10/2024**
Section: CPE21S1	**Date Submitted: 16/10/2024**
Name(s): ELIJAH YOHANCE SILANG	**Instructor: Engr. MARIA RIZETTE SAYO**

6. Output

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <algorithm>
using namespace std;

void createRandomArray(int arr[], int size) {
srand(time(0));

        for (int i = 0; i < size; ++i) {
        arr[i] = rand() % 100;
        }
}

void printArray(int arr[], int size) {
        for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
        }

        cout << endl;

}

int main() {
const int size = 100;
int arr[size];

        createRandomArray(arr, size);
        cout << "Array: ";

        printArray(arr, size);

return 0;

}
```

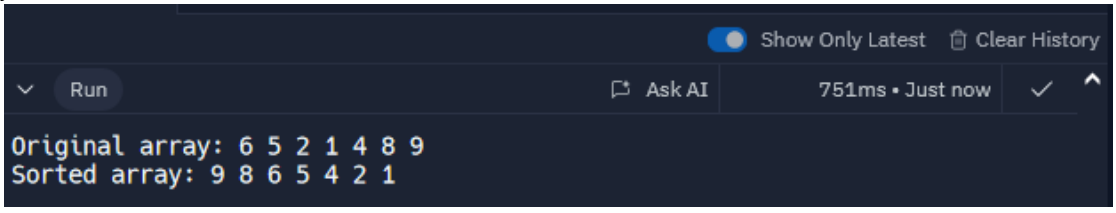
```
Run                                   Ask AI    1s • Just now

Array: 8 94 73 38 21 3 40 67 49 34 19 96 12 63 18 51 82 70 82 0 38 46 98 10 30 76 20 7
6 16 41 17 76 87 91 67 9 46 7 28 47 93 48 96 57 11 14 8 45 36 91 97 26 89 95 36 19 71
9 95 39 50 65 16 38 56 83 99 54 90 27 1 83 27 49 40 90 63 0 87 52 43 85 78 32 32 15 52
 56 76 99 95 26 64 63 16 72 98 67 26 88
```

Observation	This C++ code generates a random array of 100 integers. By creating a randomized array, the code provides a diverse range of input data for evaluating the correctness of sorting algorithms. This is very useful for testing

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```
#include <iostream>
#include <algorithm>
template <typename T>

void bubbleSort(T arr[], size_t arrSize) {
        for ( int i = 0; i < arrSize; i++) {

        for (int j = i + 1; j < arrSize; j++) {
        if (arr[j] > arr[i]) {
        std::swap(arr[j], arr[i]);
        }
        }
        }
}
int main() {
int arr[] = {6, 5, 2, 1, 4, 8, 9};
size_t arrSize = sizeof(arr) / sizeof(arr[0]);

        std::cout << "Original Array: ";

        for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
        }

        std::cout << std::endl;

bubbleSort(arr, arrSize);

        std::cout << "Sorted Array: ";

        for (int i = 0; i < arrSize; i++) {
        std::cout << arr[i] << " ";
        }

        std::cout << std::endl;

return 0;
}
```



Observation	The C++ code rearranges the given integer values inside the Original Array to a sorted one inside the Sorted Array starting from highest value to lowest. This will be useful for multiple inputs of random integers that a person alone cannot do manually.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

```
#include <iostream>
template <typename T>

int Routine_Smallest(T arr[], int K, const int arrSize) {
        int position = K;
        T smallestElem = arr[K];

        for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
        smallestElem = arr[j];
        position = j;
        }
        }
return position;
}

template <typename T>

void selectionSort(T arr[], const int N) {
        for (int i = 0; i < N - 1; i++) {
        int POS = Routine_Smallest(arr, i, N);
        std::swap(arr[i], arr[POS]);
        }
}

int main() {
int arr[] = {6, 5, 2, 1, 4, 8, 9};
const int N = sizeof(arr) / sizeof(arr[0]);

        std::cout << "Original array: ";

        for (int i = 0; i < N; i++) {
        std::cout << arr[i] << " ";
        }

        std::cout << std::endl;

selectionSort(arr, N);

        std::cout << "Sorted array: ";

        for (int i = 0; i < N; i++) {
        std::cout << arr[i] << " ";
        }

        std::cout << std::endl;
```

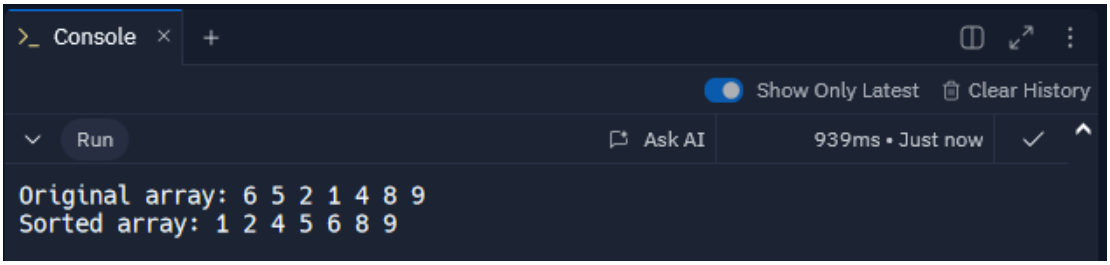
	return 0; } 
Observation	The C++ code arranges the given integer values inside the Original Array into a sorted one inside the Sorted Array, starting from the lowest value to the highest. This is useful for multiple inputs of random integers that a person alone cannot manually sort.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	#include <iostream>
	template <typename T>
	
	void insertionSort(T arr[], const int N) {
	for (int K = 1; K < N; K++) {
	T temp = arr[K];
	int J = K - 1;
	
	while (J >= 0 && arr[J] > temp) {
	arr[J + 1] = arr[J];
	J--;
	}
	
	arr[J + 1] = temp;
	}
	}
	
	int main() {
	int arr[] = {6, 5, 2, 1, 4, 8, 9};
	const int N = sizeof(arr) / sizeof(arr[0]);
	
	std::cout << "Original array: ";
	
	for (int i = 0; i < N; i++) {
	std::cout << arr[i] << " ";
	}
	
	std::cout << std::endl;
	
	insertionSort(arr, N);
	
	std::cout << "Sorted array: ";
	
	for (int i = 0; i < N; i++) {

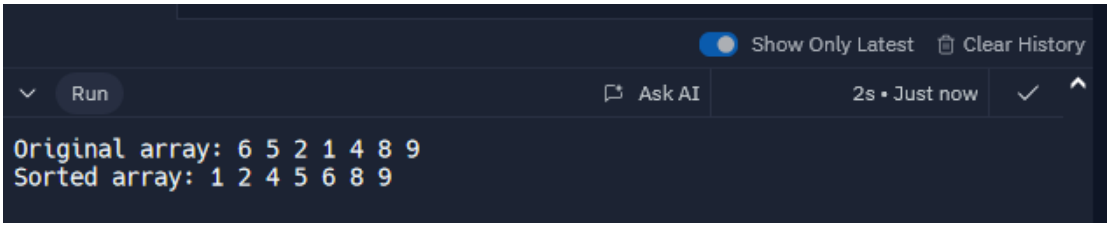
	std::cout << arr[i] << " "; } std::cout << std::endl; return 0; } 
Observation	Insertion sort works by iterating through an array, taking each element and placing it in its correct position within the sorted portion of the array. This involves shifting elements to the right until the correct spot is found for the current element. It's like sorting a deck of cards by picking up cards one by one and inserting them into their correct position in the sorted pile.

Table 7-4. Insertion Sort Algorithm

7. Supplementary Activity

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm

8. Conclusion

9. Assessment Rubric